ASSIGNMENT-2

Name: P. Greeshma

Regno: 192424418

Course Code: DSA0120

Github Link : https://github.com/parvathareddygreeshma-png/DSA0120-Objected-programming-with-c-

Submission date: 09-11-2026

Q1 – Call by value, reference, and pointer

Predict the given program's output before running it:

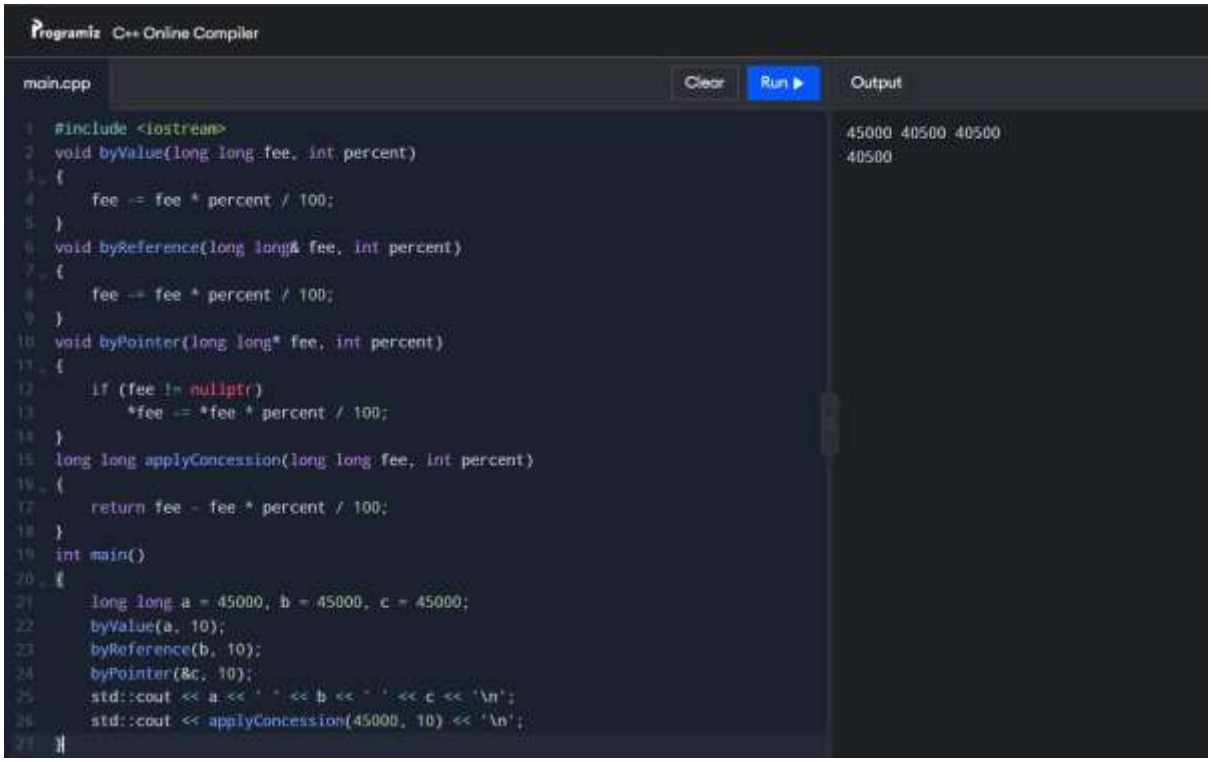
1 12 13

1 100

16 13 6

a is passed by value, so it stays 1. Reference b and pointer c change the original variables. bigger() returns a reference to the larger variable, so assigning 100 changes c. The show() calls use default values for omitted arguments.

Implementation:



Design choice: Returning the new fee is clear when chaining several concessions. A reference is useful when the function should update the student's stored fee directly. A const reference is for reading a large object without copying or changing it.

All approaches take **O(1)** time. Passing a long long copies only a small value; passing a large Student object by value copies the whole object, while a reference avoids that copy.

Q2 – Returning references



Output: 94. Returning a reference to an array element lets the caller modify it. Returning a reference to a local variable is unsafe because that variable no longer exists after the function returns; the compiler usually warns about it, and using it is undefined behaviour.

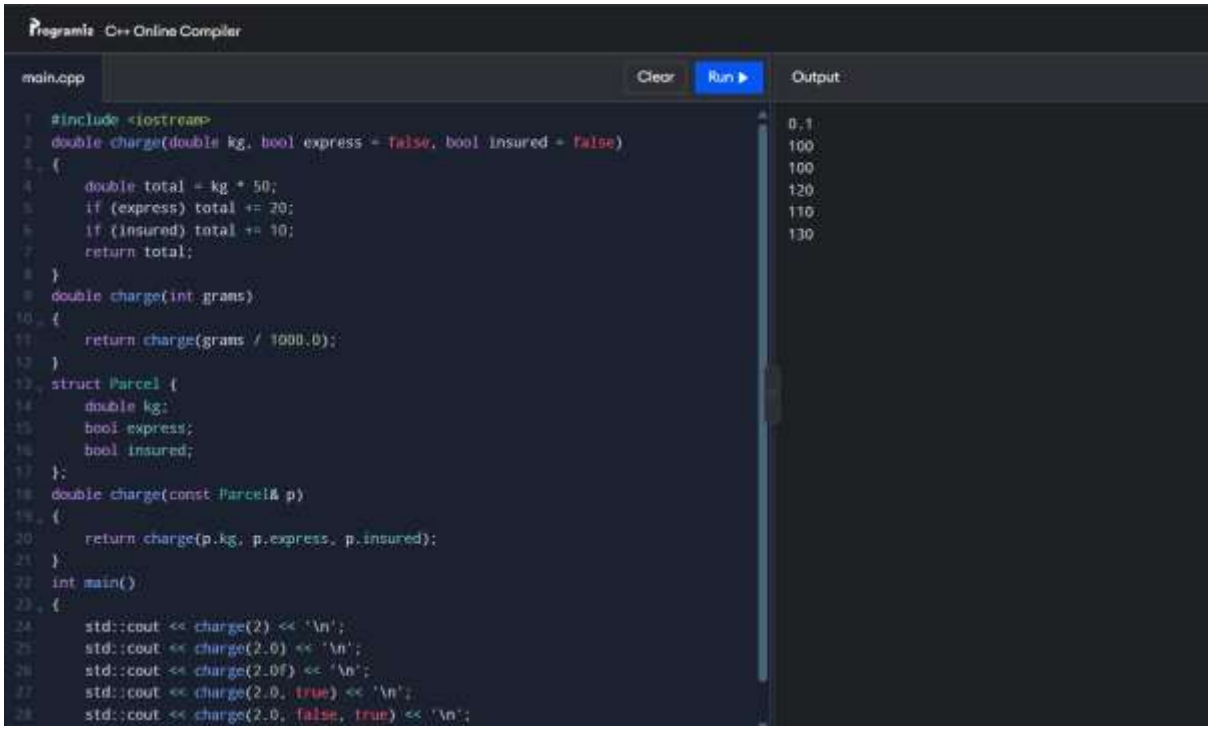
A const reference allows reading without copying but prevents modification. Returning a copy is safe but changing it won't update the array. Returning an index is often safer for a growing vector, where resizing may invalidate references and pointers.

Finding the maximum takes O(n) time and O(1) extra space.

Q3 – Default arguments and overloading

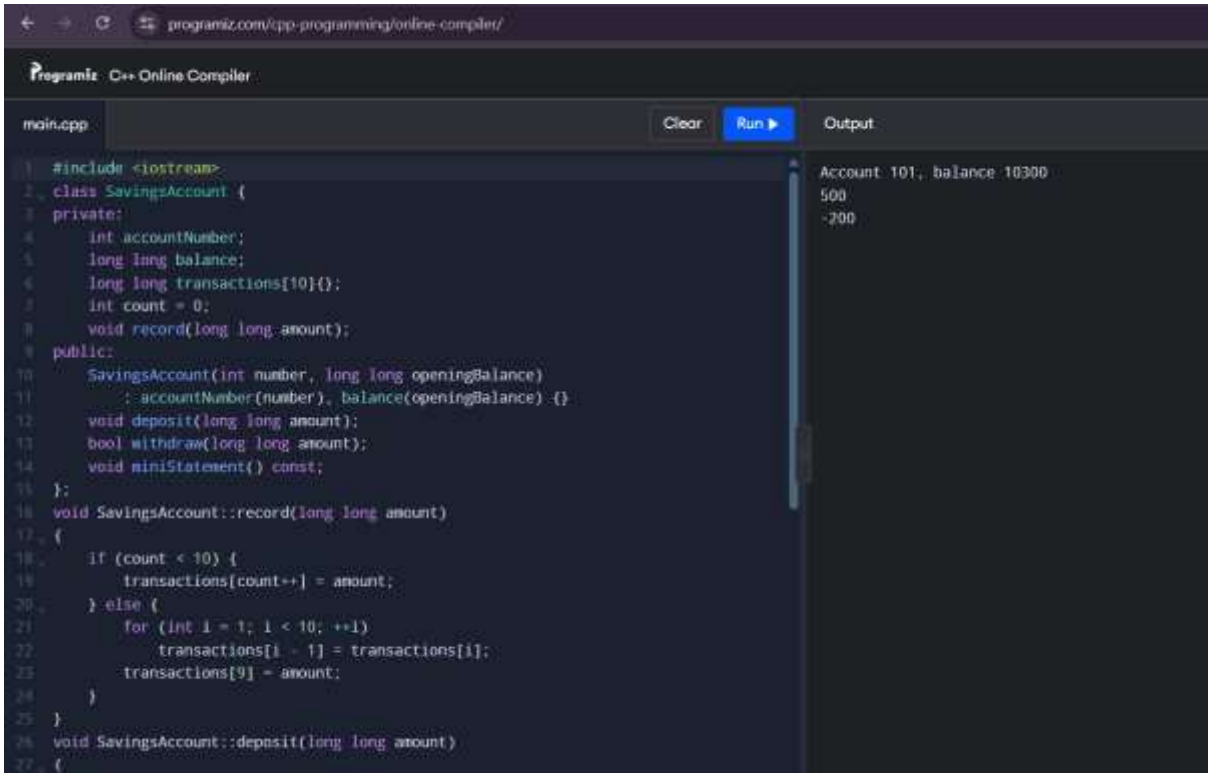
The original functions are ambiguous because calls such as charge(2) match multiple overloads. Keep one function with defaults, and overload only for distinct input types.

Overload selection prefers an exact match, then a promotion, then a conversion. Default arguments must be supplied from the right. Overload selection happens at compile time, so it adds no runtime cost; each call takes O(1) time.

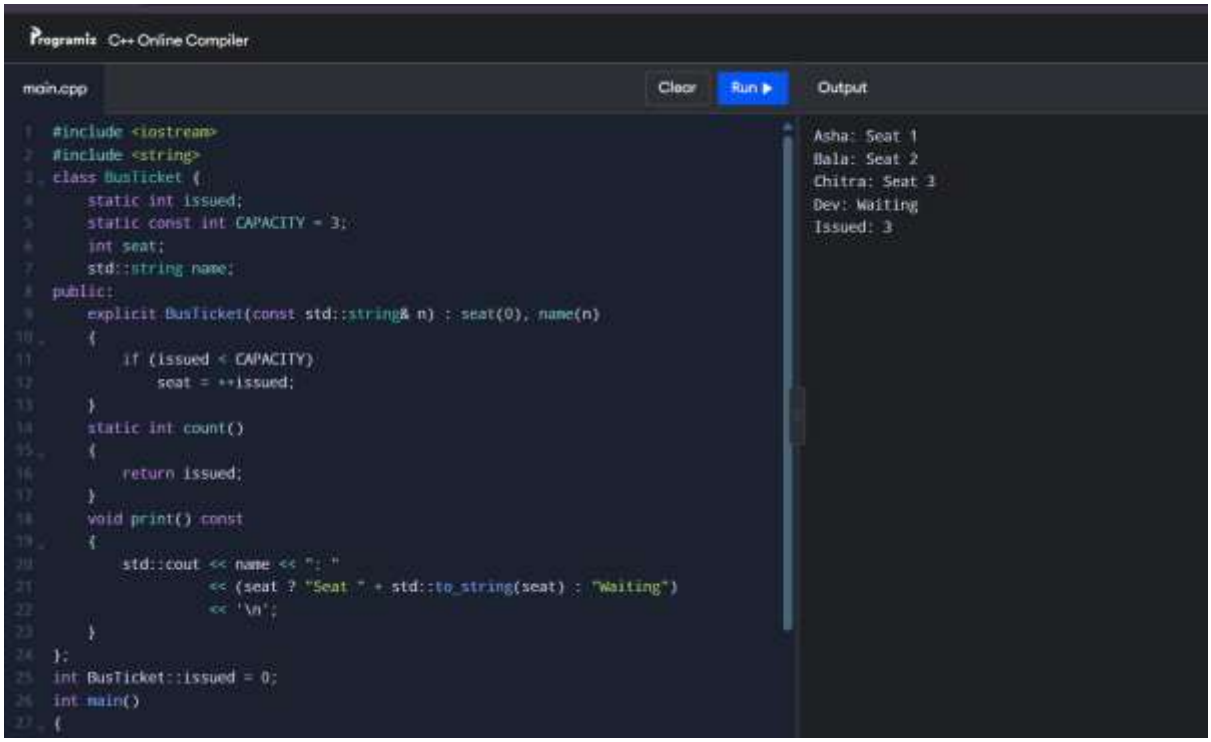


Q4 – Savings account with the last 10 transactions

The array is private so transactions and balance can only change through checked functions. Shifting keeps the newest 10 entries but costs O(10) per new transaction; a circular buffer adds entries in O(1). Changing the capacity to 50 is easiest with a named constant or a dynamically sized container.



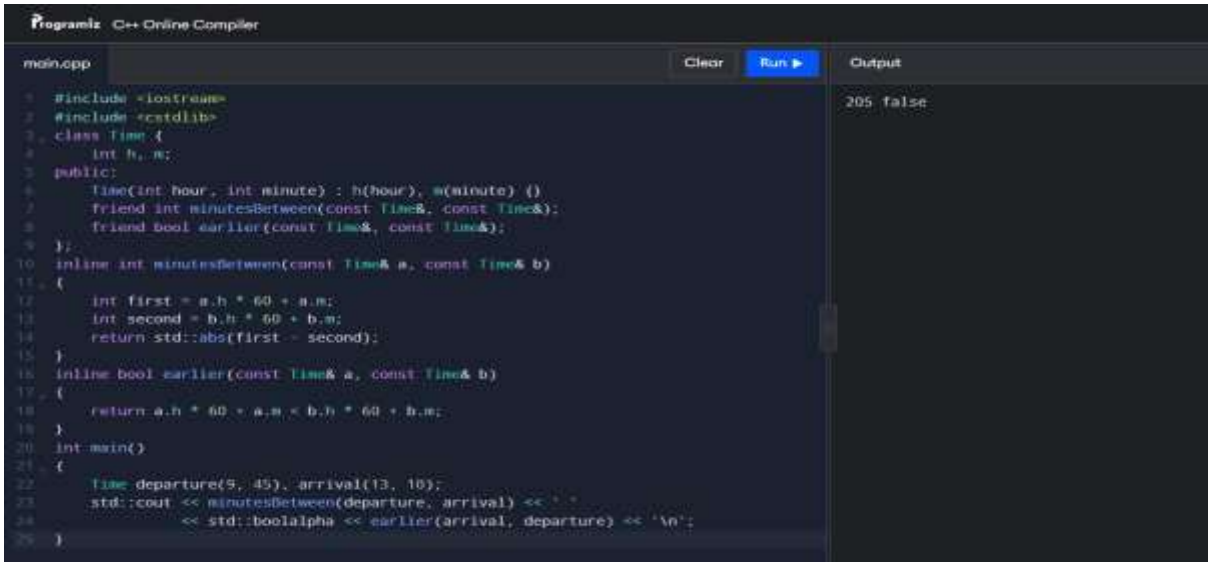
Q5 – Static members and bus tickets



issued is static because it is shared by all tickets; its definition is outside the class. A static function has no this object, so it can't access non-static members. The counter persists across function calls. For two buses, use a separate counter per bus (such as a TicketOffice object) instead of one class-wide counter.

Q6 – Friend functions and inline

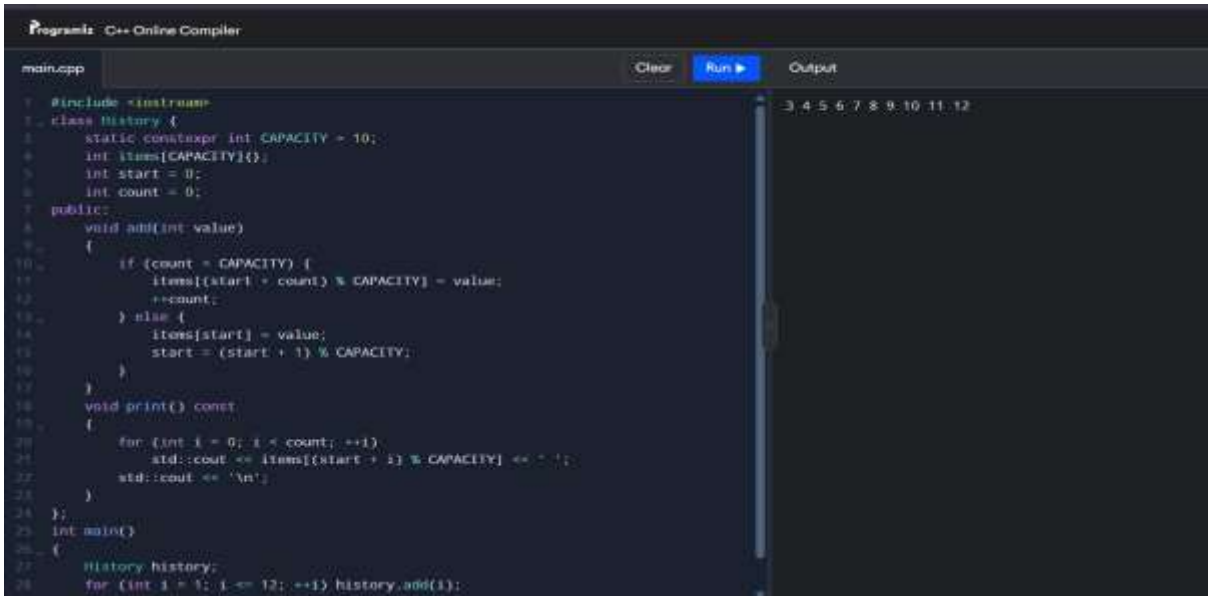
Friend functions are called like ordinary functions but can access private members. inline permits definitions in headers and suggests inlining; the compiler decides whether to do it. These functions take O(1) time. To complete the assignment, test equal times and times across noon, and measure 10 million calls with -O0 and -O2.



Q7 – Redesign the transaction history

Limitation: Q4 shifts the whole array whenever a new transaction arrives after it fills. With capacity k, that takes O(k) work.

Objective and prediction: Keep the latest 10 transactions while recording each new one in O(1) time. Test 12 entries; expect to retain entries 3–12.



The circular buffer uses O(1) space for a fixed capacity and O(1) work per addition; shifting uses O(k) work. Run both versions with the same large set of transactions and record timings to verify the improvement. For small histories, shifting may still be simpler.